# Assignment 2

*Abdalrahman Mohammed*
*abmm22@student.bth.se*

## LSTM Autoencoder-Based Anomaly Detection and Baseline Model Comparison

### Introduction

Anomaly detection is used to find patterns that are different from normal behaviour. Examples include system faults, unusual activity or rare events. In this project anomaly detection is applied to the NASA SMAP/MSL anomaly detection dataset which contains time-series data from spacecraft systems.

Since the data is time-dependent LSTM autoencoders are suitable because they can learn normal patterns over time. The models reconstruct input sequences and sequences with high reconstruction error are classified as anomalies. The aim of this project is to implement and compare two LSTM autoencoder models. A K-Means clustering model is also used as a simpler unsupervised baseline. The models are evaluated using Precision, Recall, F1-score, ROC-AUC and visualizations of detected anomalies over time.

### Data Preprocessing

In this project only one data channel A-1 was selected to keep the experiment focused and easier to run. This channel contains 2880 training time steps and 8640 test time steps. Each time step has 25 features.

The training data was treated as normal system behaviour which is the standard assumption for this dataset. The test data contains both normal and abnormal behaviour. The file labeled_anomalies.csv was used to create the true anomaly labels for channel A-1. These labels show where the real anomalies are in the test data.

The first preprocessing step was checking for missing values. The selected channel did not have any missing values. But the code still included this step to meet the preprocessing requirement. Missing values were handled with forward fill and backward fill. If any missing values were still left they were replaced with zero.

After the features were scaled using StandardScaler. The scaler was fitted only on the training data. Then the same scaler was used on both the training and test data. This avoids data leakage because the test data should not influence preprocessing. Scaling was used to make the features more similar in size which helps both the LSTM autoencoders and the K-Means model.

The time series data was then changed into fixed length sequences so it could be used by the LSTM models. A sequence length of 50 was used. This means that each sample contains 50 time steps and each time step has 25 features. After this step the training data shape was (2831, 50, 25) and the test data shape was (8591, 50, 25). The training data was then split into

training and validation sets using an 80/20 split. The validation set was later used to choose the anomaly threshold.

The original test labels were made for each time step. Each time step was labelled as normal or anomalous. Since the models give one anomaly score for each sequence the labels also had to be changed to sequence level labels. Each sequence got the label of its last time step. This made the labels match the reconstruction errors used for evaluation and plotting.

## Model Design

Two different LSTM autoencoder models were used. Both models follow the same basic idea. The encoder compresses the input sequence into a smaller representation. The decoder then tries to reconstruct the original sequence from it. Since the models were trained mostly on normal data they should reconstruct normal sequences better than anomalous sequences.

The first model was a simple LSTM autoencoder. It had one LSTM encoder layer, one fully connected latent layer, one LSTM decoder layer and one output layer. The hidden size was 64 and the latent size was 32. This model was used as a small baseline model. It has fewer parameters and trains faster which makes it useful for comparison with the deeper model.

The second model was a deeper LSTM autoencoder. It had two LSTM layers in the encoder and two LSTM layers in the decoder. The hidden size was increased to 128 and the latent size was increased to 64. Dropout of 0.2 was also used. This model was larger than the simple model so it could learn more complex patterns in the data. The purpose of comparing the two models was to see if a deeper model gives better anomaly detection results.

## Training Configuration

Both LSTM autoencoders were trained by reconstruction. This means that the input sequence was also the target. The model received a sequence and tried to recreate the same sequence as accurately as possible.

The loss function used for both models was Mean Squared Error. Mean Squared Error measures the difference between the original input sequence and the reconstructed sequence. A low reconstruction error means the model reconstructed the sequence well. A high reconstruction error means the sequence may be different from the normal patterns learned during training.

The optimizer used was Adam with a learning rate of 0.001. The batch size was 64 and both models were trained for 50 epochs. The sequence length was 50 and each time step had 25 features. The validation set was used to check the reconstruction loss during training. It was also used later to choose the anomaly thresholds.

The training loss and validation loss were plotted for both models. These plots show how well each model learned to reconstruct normal sequences during training.


Figure 1. Training and validation loss for the simple LSTM autoencoder.

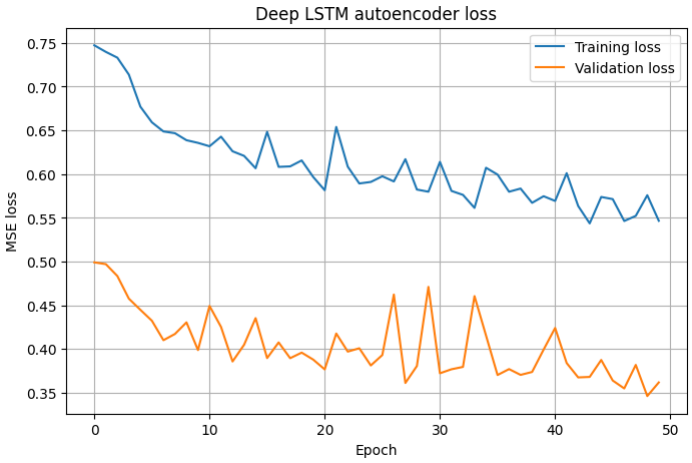


Figure 2. Training and validation loss for the deep LSTM autoencoder.

The loss curves showed that both models reduced the reconstruction error during training. This means that both LSTM autoencoders learned patterns from the normal training data.

## Anomaly Detection Strategy

Anomalies were detected using reconstruction error. After training each model reconstructed the test sequences. Then the reconstruction error was calculated for each sequence by comparing the original sequence with the reconstructed sequence. The error was calculated across all time steps and features.

The main idea is that normal sequences should have low reconstruction error because the models were trained on normal behaviour. Anomalous sequences should be harder to reconstruct so they should get higher reconstruction error.

A threshold was used to classify the reconstruction errors. If a sequence had an error above the threshold, it was classified as anomalous. If the error was below the threshold, it was classified as normal.

The thresholds were calculated from the validation reconstruction errors using different percentiles: 90, 92, 95, 97, 98 and 99. Each threshold was then tested on the test reconstruction errors. The results were compared using precision, recall and F1-score. F1-score was used because the dataset is highly imbalanced. This means that there are many more normal sequences than anomalous sequences. Accuracy can therefore be misleading because a model could classify almost everything as normal and still get high accuracy. F1-score is more useful because it balances precision and recall.

Based on the threshold comparison table the final thresholds were chosen manually. The simple LSTM autoencoder used the 98th percentile threshold. The deep LSTM autoencoder used the 97th percentile threshold. These thresholds gave the highest F1-score for each LSTM model.

## Evaluation

The models were evaluated using reconstruction error, Precision, Recall, F1-score and ROC-AUC. Precision shows how many detected anomalies were correct. Recall shows how many real anomalies the model found. F1-score combines precision and recall into one score. ROC-AUC shows how well the model separates normal sequences from anomalous sequences using the anomaly scores.

The test set was highly imbalanced. There were 8591 test sequences but only 85 of them were anomalies. This means that anomalies made up less than 1% of the test data. Because of this accuracy was not used as the main metric. A model could classify almost everything as normal and still get high accuracy even if it misses the anomalies. The results for the two LSTM autoencoders were:

| **Model** | **Threshold** | **Precision** | **Recall** | **F1-score** | **ROC-AUC** |
|---|---|---|---|---|---|
| Simple LSTM autoencoder | 98th percentile | 0.0694 | 0.3412 | 0.1153 | 0.8671 |
| Deep LSTM autoencoder | 97th percentile | 0.0599 | 0.5176 | 0.1073 | 0.8233 |

The simple LSTM autoencoder achieved the highest F1-score and ROC-AUC. This means it had the best overall balance between precision and recall. It also separated normal and anomalous sequences better overall. The deep LSTM autoencoder had higher recall meaning it found more true anomalies. It also detected more false positives which gave it slightly lower precision and F1-score.

Both LSTM models were able to detect some anomalies using reconstruction error. The low precision shows that many detected anomalies were false positives. This is a common problem in unsupervised anomaly detection when anomalies are very rare.

## Visualization

The reconstruction error plots were used to show the anomaly detection results over time. Each plot shows the reconstruction error for the test sequences, the selected threshold, the true anomalies and the detected anomalies.

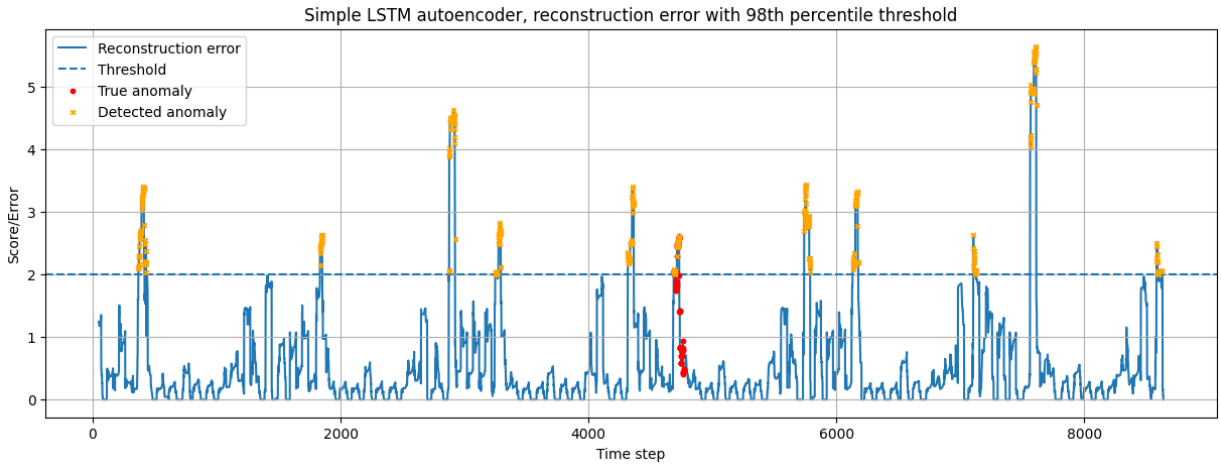


Figure 3. Reconstruction error over time for the simple LSTM autoencoder using the selected threshold.

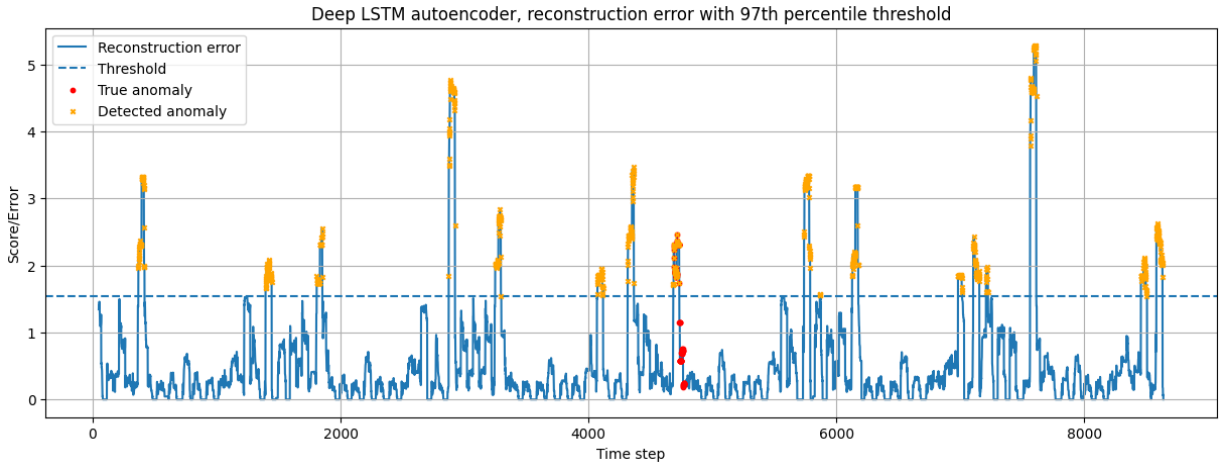


Figure 4. Reconstruction error over time for the deep LSTM autoencoder using the selected threshold.

The plots show that both LSTM models had clear spikes in reconstruction error at several time points. Some of these spikes matched the labelled anomaly intervals but some did not. This explains why the models had moderate recall but low precision. The models could find unusual patterns in the test data but not every high reconstruction-error spike was a real labelled anomaly.

## Model Comparison

The two LSTM autoencoder models were compared with each other and with a K-Means baseline. K-Means was used because it is a simple and common unsupervised method. It groups similar data points into clusters. K-Means was trained with 5 clusters and n_init = 20. The n_init value means that K-Means was run several times with different initial cluster positions and the best result was kept.

To make the comparison fair K-Means used the same preprocessed sequence data as the LSTM models. However K-Means needs two dimensional input so each sequence with shape [50, 25] was flattened into one vector of length 1250. This allowed K-Means to use the same sequence windows as the LSTM models. For K-Means the anomaly score was calculated as the distance from each test sequence to the nearest cluster center. A larger distance means that the sequence is more different from the normal training data so it is more likely to be anomalous.

The K-Means threshold values were calculated from the validation anomaly scores using the same percentile thresholds as the LSTM models: 90, 92, 95, 97, 98 and 99. These thresholds were then compared using the labelled test set to report precision, recall and F1-score. Based on the threshold comparison table the 97th percentile threshold was used for K-Means because it gave the highest F1-score.

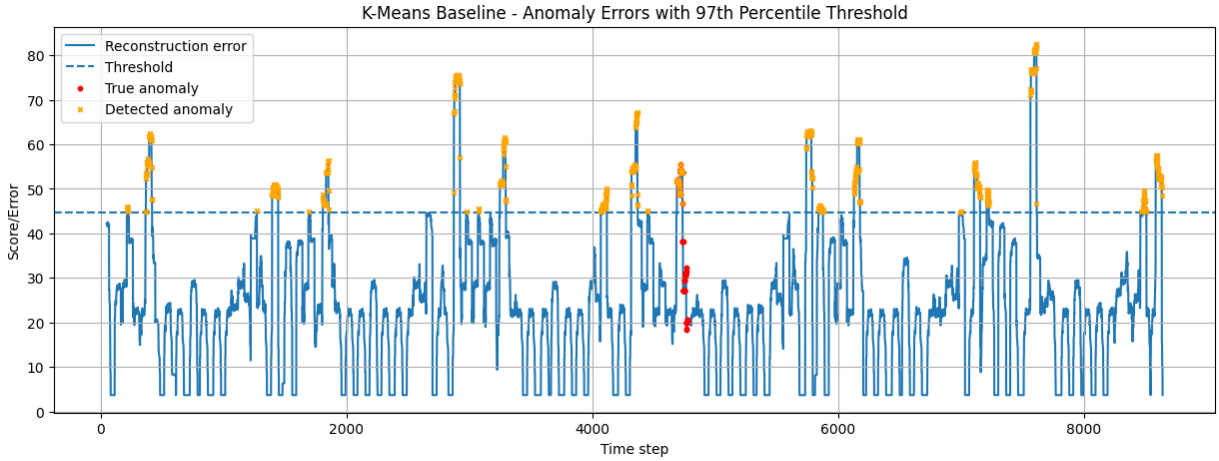


Figure 5. K-Means anomaly scores over time using the selected threshold.

The final comparison between all three models:

| Model | Threshold | Precision | Recall | F1-score | ROC-AUC |
|---|---|---|---|---|---|
| Simple LSTM Autoencoder | 98th percentile | 0.0694 | 0.3412 | 0.1153 | 0.8671 |
| Deep LSTM Autoencoder | 97th percentile | 0.0599 | 0.5176 | 0.1073 | 0.8233 |
| K-Means Baseline | 97th percentile | 0.0589 | 0.5176 | 0.1058 | 0.7970 |

The simple LSTM autoencoder had the best F1-score and the highest ROC-AUC. The deep LSTM autoencoder and K-Means had higher recall which means they found more true anomalies. They also produced more false positives which lowered their precision.

K-Means performed almost as well as the deep LSTM model in recall and F1-score. Both LSTM models had higher ROC-AUC values. This means that the LSTM reconstruction errors were better at separating normal and anomalous sequences overall than the K-Means distance score. The main weakness of K-Means is that it does not model the temporal order inside each sequence while the LSTM autoencoders can use the sequential structure of the data.

The simple LSTM autoencoder was the best overall model in this experiment. It gave the best balance between detecting anomalies and separating normal sequences from anomalous ones. The deep LSTM model was larger but it did not perform better on this selected channel.

## Discussion

The results show that LSTM autoencoders can be used for anomaly detection in sequence data. LSTM models are useful because they can learn patterns over time. This makes them suitable for spacecraft sensor data. The reconstruction error method is also easy to understand. If a model is trained on normal behaviour and reconstructs a sequence poorly  the sequence may be abnormal.

The results also show some limitations. The main limitation is the low precision. The models detected some true anomalies but many predicted anomalies were false positives. This is because the dataset is highly imbalanced. Less than 1% of the test sequences were anomalies so even a small number of false positives can lower precision a lot.

Another limitation is that only one data channel A-1 was used. Different channels in the NASA SMAP/MSL dataset may have different anomaly patterns. A model may perform differently on another channel. Therefore, the results should only be seen as results for the selected channel, not for the whole dataset.

The threshold method is also a limitation. Percentile based thresholds are simple and useful but the results depend a lot on the chosen threshold. A higher threshold can improve precision but it may miss more true anomalies. A lower threshold can find more anomalies but it can also create more false positives. In a real system the best threshold would depend on what is worse for example false alarms or missed anomalies.

The comparison with K-Means shows that simpler baseline methods can still perform well in some metrics. K-Means does not directly learn patterns over time. It treats each sequence as one long vector while the LSTM models use the time order of the sequence. This is probably why the LSTM models achieved better ROC-AUC values.

In real world anomaly detection these models could be useful as early warning systems. However they should not be used to make fully automatic decisions without human review. False positives can happen, and some real anomalies may also be missed. A real system would need better threshold tuning, testing on several channels, validation by domain experts and possibly a combination with other detection methods.

## Reproducibility

To reproduce the results download the NASA SMAP/MSL dataset from Kaggle: https://www.kaggle.com/datasets/patrickfleith/nasa-anomaly-detection-dataset-smap-msl. After downloading unzip the dataset. Then take the data folder and the labeled_anomalies.csv file and place them in the same folder as the notebook. Do not run the requirements cell. This cell is only included to show the Python and package versions used in the environment. Start running the notebook from the imports cell and then run all cells below it in order from top to bottom. The notebook already contains the selected channel, model settings, preprocessing steps, training procedure, threshold selection, evaluation and plots needed to reproduce the results.

## Conclusion

This project used two LSTM autoencoder models and one K-Means baseline for anomaly detection on the NASA SMAP/MSL dataset. The preprocessing steps included handling missing values, scaling the features and creating fixed-length sequences for the LSTM models. Both LSTM autoencoders were trained to reconstruct input sequences. Anomalies were then detected using reconstruction error thresholds calculated from the validation data and compared using the labelled test set.

The simple LSTM autoencoder had the best overall performance. It achieved the highest F1-score and ROC-AUC. The deep LSTM autoencoder had higher recall meaning it found more anomalies. It also produced more false positives which gave it a slightly lower F1-score. The K-Means baseline performed well in recall and F1-score but its ROC-AUC was lower than both LSTM models.

The results show that LSTM autoencoders can be used for sequence based anomaly detection. The performance depends a lot on the chosen threshold and the class imbalance in the data. The experiment also shows that deep learning models should be compared with simpler baseline models. This helps show whether the more complex model improves performance.